

Simulation-based Robot Skill Learning System using PPO and Cauchy-Distribution-based Reward Design

1st Shun ZHANG

Robot Control Team

Shanghai Electric Group Central
Academe

Shanghai, China

zhangshun9866@163.com

2nd Kaifeng JIAN

Robot Control Team

Shanghai Electric Group Central
Academe

Shanghai, China

kaifeng.jian@outlook.com

3rd Zejun SUN

Robot Control Team

Shanghai Electric Group Central
Academe

Shanghai, China

sunzejunvc@sina.com

Abstract—In recent years, the application of robots in the industrial sector has been steadily increasing, leading to a gradual rise in the workload of robot development. However, the cost of conducting physical experiments for functional development tailored to different scenarios is high and time-consuming. In this paper, we propose a robot skill learning structure to enable robot to learn specific skills automatically without user programming in a simulation scenario. The proposed structure consists of a skill learning platform and a skill learning toolbox. The skill learning platform is designed to construct simulation scenarios for skill learning for various types of robots. The skill learning toolbox is designed to provides necessary tools for simulation and skill learning using Docker. The Proximal Policy Optimization (PPO) algorithm is served as skill learning algorithm. A Cauchy-distribution-based reward design method is proposed to select the optimal influence coefficient of each reward term and then to obtain the highest reward. Under this robot skill learning structure, robot can successfully learn a skill in a simulation scenario through our experiments. Additionally, using this skill learning structure can reduce the time of development, and increases the reusability rate of the skill.

Keywords—Robot Skill Learning System; Skill Learning Platform; Skill Learning Toolbox; Proximal Policy Optimization (PPO) algorithm; Cauchy-Distribution-based Reward Design

I. INTRODUCTION

As manufacturing demand increases and the population ages, the replacement of humans by machines has become an inevitable trend. Many assembly operations show a high potential for robot-based automation but are not realized in industrial applications. One reason is the time-consuming and complex programming of the required position/force control or vision servo systems to adapt to process deviations. Thus, easy-to-use and efficient tools for robot programming are highly desirable. A promising approach to ease the programming burden on the user is skill learning [1, 2, 3]. Skills provide reusable motion control and plan modules for similar tasks, capsulizing the relationship between robot control and sensor observations, and can be parameterized to realize the desired industrial task. In contrast to standard programming methods, robot skills can be transferred to different types of robots as they rely on task and not the robot-specific kinematic model. Despite their advantages, it still takes some time to parameterize and tune the skills manually on the physical robot system. Besides,

considerable financial support is necessary to establish special laboratories for learning various kinds of skills. These factors limit their usage in industrial fields. An alternative approach is simulation-based offline programming. Unlike online methods, this does not need the use of a physical robot cell. However, the control programs must be created manually and later fine-tuned to address the differences between the virtual and real world.

A. Related work

To achieve the goal of enabling robots to learn one or more skills within a simulation environment using a robot training framework, the academic community has widely focused on end-to-end robot learning technologies. Whether through imitation learning or reinforcement learning, these approaches typically involve collecting task-specific data in either single-task [4, 5] or multi-task [6, 7] settings, where the data is meticulously tailored to the tasks the robot is expected to perform. However, building a universal multi-task robot training framework is fraught with challenges. Although Reed et al. [8] have proposed several large-scale multi-task robot policies in recent years, such models often have limited applicability in real-world tasks, as seen with Gato, or they prioritize training tasks over generalization to new tasks, as demonstrated by Shridhar et al. [9, 10] with their instruction-following methods, or Jang et al. [7] who achieved relatively lower performance on new tasks. The two main challenges in robot skill training lie in constructing suitable datasets and designing effective models. On one hand, good generalization of models requires datasets to be both large in scale and broad in scope, covering a variety of tasks and environments. On the other hand, effective multi-task learning for robots demands a robust model, and Transformer models [11] have shown exceptional performance in this regard, especially when learning numerous tasks. Some studies have proposed Transformer-based robot control strategies, such as using Transformer-processed language commands as a robust framework for specifying and generalizing to new tasks [12, 13]. From the perspective of task content, skill learning for grasping [5] and peg insertion [5, 26] has been the most extensively researched.

Numerous excellent simulators have made significant contributions to the field of robot simulation, such as Isaac Sim, Isaac Gym, Pybullet, MuJoCo, CoppeliaSim, and Gazebo [14, 15]. NVIDIA Isaac Sim, a robot simulator based on Omniverse,

has been used by Mayank Mittal et al. [16] and Serrano-Muñoz et al. [17] to study the creation of robot learning environments using reinforcement learning. The platform also supports other

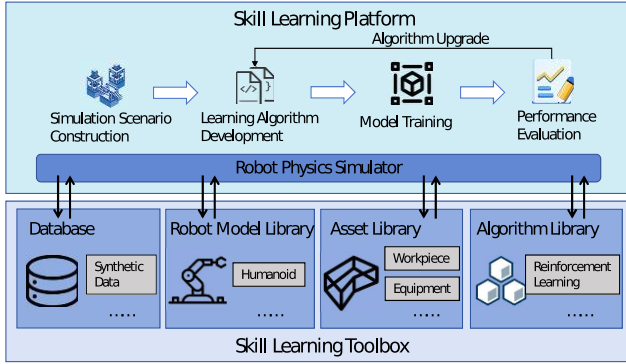


Fig. 1. Overview of the robot skill learning structure

robot learning algorithms such as imitation learning and transfer learning. Isaac Gym, on the other hand, is NVIDIA's specialized physics simulation environment for reinforcement learning, with IsaacGymEnvs providing reinforcement learning environments as examples [18]. MuJoCo, an open-source physics simulation engine, is primarily used for fast and accurate simulations, with Andrychowicz et al. [19] achieving dexterous hand control on this platform. Pybullet is mainly used for mobile robot simulations, with Tan et al. [20] utilizing the platform to transfer mobile strategies for quadruped robots from simulation to reality. CoppeliaSim (formerly V-Rep) is a popular choice for simulating ground mobile robots. Recent studies have used CoppeliaSim for biped robot navigation planning [21], differential drive robot navigation planning [22], and visual trajectory tracking for differential drive robots [23]. Gazebo is widely used in mobile ground robot research, covering both legged [24] and wheeled [25] robots. The Robot Operating System (ROS) interface provided by Gazebo simplifies the process for developers to test programs in simulation and facilitates migration to real physical systems.

B. Contribution

In this paper, we propose a robot skill learning system for robots in different tasks. It consists of a robot skill learning platform and a skill learning toolbox. Robots can learn the necessary skills in this skill learning platform, and skill learning toolbox provides necessary tools for data and robot model storage, simulation scenario construction, and skill learning algorithm. The main contributions of this paper are as follows:

- (1) A skill learning platform is proposed to provide simulation scenarios for skills training for various types of robots. It can reduce development time, experimental costs, and improve the reusability of robot skills compared with using real robots and setting up real experimental environments.
- (2) A Docker-based skill learning toolbox is designed to provides necessary tools for data and robot model storage, simulation assets library, and skill learning algorithm library. It can accelerate the simulation scenario construction, and robot skill learning compared with conventional methods.
- (3) A Cauchy-distribution-based reward design method is proposed to select the influence coefficient of each reward term.

This coefficient term can be updated automatically in order to obtain the highest reward. This can help the PPO-based skill learning process more fast and efficiently.

II. ROBOT SKILL LEARNING SYSTEM

The system structure is shown in Fig. 1. The system consists of skill learning platform and skill learning toolbox. The skill learning platform is the foundation of the system. A robot physics simulator is necessary for the platform so as to acquire accurate physics simulation. There are four steps in robot skill learning: simulation scenario construction, learning algorithm development, model training, and performance evaluation. All four steps are conducted in the skill learning platform. The second part in the system is the skill learning toolbox. It supplies necessary tools for skill learning, includes database, robot model library, asset library, and algorithm library.

A. Robot Physics Simulator

The operation of robotic arms exists a series of challenging tasks, including grasping, transporting, placing, pushing, peg inserting, and so on [26]. To accurately simulate various task scenarios in a virtual environment and enable robots to learn diverse skills, it is essential to select an appropriate physics simulator. Currently, there are numerous simulators available for robotics simulation, such as Isaac Sim, Isaac Gym, Pybullet, MuJoCo, CoppeliaSim, and Gazebo, among others. The characteristics of these simulators [14, 15] can be summarized in Table 1. We compare these common simulators from four perspectives: the physics engine used, the types of sensors supported, whether they support GPU-accelerated simulation, and the number of academic citations. The statistics for academic usage are current as of December 20, 2023. Both Isaac Sim and Isaac Gym support the proprietary PhysX 5 physics engine, with Isaac Gym additionally supporting Flex. Pybullet and MuJoCo support Bullet and MuJoCo, respectively, while CoppeliaSim and Gazebo support multiple simulators. In terms of sensors, all these simulators support RGBD images and force sensors. Isaac Sim supports the most sensors, including LiDAR, IMU, and contact sensors. Pybullet and MuJoCo support tactile sensing. Regarding GPU-accelerated simulation, Isaac Sim and Isaac Gym have a significant advantage over other simulators, as both support GPU acceleration, which greatly speeds up the training process of reinforcement learning, making the industrial application of reinforcement learning in robotics control a reality. However, from the perspective of academic citations, the older and open-source MuJoCo is more favored. Although Isaac Sim has many attractive features, its recent introduction, difficulty of use, and high hardware requirements have resulted in limited usage so far. Isaac Gym, on the other hand, has a better usage rate than Isaac Sim, indicating that it is easier to use.

Overall, since Isaac Gym supports multi-robot parallel accelerated training, reinforcement learning algorithms, and can simulate RGBD and force sensors, it is suitable for various tasks. Compared to Isaac Sim, it has lower hardware requirements and a higher usage rate among other researchers. Therefore, we have chosen Isaac Gym as our robotics simulator.

TABLE I. OVERALL COMPARISON OF DIFFERENT PHYSICS SIMULATORS [14]

Simulator	Physics Engine	Sensor	GPU-accelerated Simulation	Popularity
Isaac Sim	PhysX 5	RGBD; Lidar; Force; Effort; IMU; Contact; Proximity	☑	/
Isaac Gym	PhysX 5, Flex	RGBD; Force; Contact;	☑	375
Pybullet	Bullet	RGBD; Force; IMU; Tactile;	☒	1942
MuJoCo	MuJoCo	RGBD; Force; IMU; Tactile;	☒	4884
Coppelia Sim	MuJoCo; Bullet; ODE; Newton; Vortex	RGBD; Force; Contact;	☒	1786
Gazebo	Bullet; ODE; DART; Simbody	RGBD; Lidar; Force; IMU;	☒	3949

B. Skill learning Platform

Skill learning Platform aims at four steps in robot skill learning: simulation scenario construction, learning algorithm development, model training, and performance evaluation. The basic function is simulation scenario construction. The setup of the simulation scenario is based on Isaac Gym, and the process is mainly divided into three major parts: setting up the scene, importing the robot, and controlling and training the robot. In the scene setup part, a blank virtual scene is first created and simulation environment parameters are set, such as whether to use GPU computing, simulation iteration cycles and counts, etc. Then, 3D assets are imported to populate the entire scene. For parts with complex structures or assets that require extremely high simulation accuracy, software like SolidWorks and 3ds Max can be used for creation; this method yields high-precision assets but is time-consuming. For assets with simple structures, such as cubes and spheres, Isaac Gym's standard asset library can be used for quick generation, reducing the time for environment setup. When importing assets, the rendering method is also set, such as convex decomposition and signed distance field (SDF), to balance rendering accuracy and computational complexity. The second part involves importing the robot body model, which can be described by URDF and XML, enhancing portability. At the same time, various parameters of the robot can be set, including joint stiffness and damping, and joint control modes such as position control and torque control can also be configured. The final part, the robot skill learning platform, supports two schemes. One scheme allows for external input to control the robot, such as using a joystick or keyboard to control the angles of each joint of the robot. The other scheme employs learning algorithms to train the robot to learn the corresponding skills according to task requirements. Currently, the main algorithms applied to robot training include reinforcement learning and imitation learning algorithms. The three parts—setting up the scene, importing the robot, and controlling and training the robot—together form the foundation of the robot virtual simulation training ground.

C. Skill learning toolbox

The construction of simulation scenarios often consumes a considerable amount of time. To efficiently set up simulation scenarios, we propose a Docker-based skill learning toolbox that enhances the efficiency of scenario construction. This method divides the scenario setup into several parallel executable parts: asset library, algorithm library, robot model library, simulation platform, and database. The 3D assets

created are stored in the asset library; various types of robot models are housed in the robot model library; robot skill learning algorithms are kept in the algorithm library; the simulation platform serves as the carrier for the simulation scenarios; and the synthetic data from the training process along with the trained skill models are stored in the database. The overall process is as follows: First, when setting up a simulation scenario, asset models from the asset library are invoked within the simulation platform to construct the scene, and robot models along with corresponding skill learning algorithms are selected to commence skill training. Subsequently, the robot training is conducted using the simulation scenarios within the simulation training platform, and finally, the obtained models are validated in the simulation environment. One advantage of using Docker is that different simulation scenarios can be built for different projects, which are independent of each other and can be switched quickly. Another advantage is that multiple users can simultaneously access and modify the same simulation scenario, with changes updated in real-time. This enables collaborative scenario construction among multiple users, significantly improving the efficiency of scenario setup.

III. SKILL LEARNING ALGORITHM

Proximal Policy Optimization (PPO) algorithm is served as the skill learning algorithm in our platform, and a Cauchy-distribution-based reward design method has been proposed to facilitate effective robotic skill training.

A. Reinforcement Learning

The problem is modeled as a Markov Decision Process (MDP), wherein the agent, at each discrete time step t , finds itself in a state $s_t \in \mathcal{S}$, executes an action $a_t \in \mathcal{A}$, and receives a scalar reward $r_t(s_t, a_t)$. The environment then transitions to the next state according to the transition probability $p(s_{t+1}|s_t, a_t)$.

$$R = \sum_{t=0}^H \gamma^t r_t \quad (1)$$

Where, H represents the planning horizon, and $\gamma \in (0,1)$ is the discount factor. In reinforcement learning, the agent learns a policy $\mathbf{a}_t = \pi_{\theta}(\mathbf{s}_t)$ from data.

B. Proximal Policy Optimization

Proximal Policy Optimization (PPO), which performs on par with or better than state-of-the-art methods, is also easier to implement and tune. Due to its user-friendliness and robust performance, the PPO algorithm has become the default reinforcement learning algorithm at OpenAI [28]. The specific steps of the PPO algorithm are based on improvements to the policy gradient methods and mainly include the following key steps:

(1) Data Collection: A set of interaction data is collected by executing the current policy in the environment. This data includes states, actions, rewards, and possible next states.

(2) Advantage Estimation: The advantage function $A^{\theta^k}(s_t, a_t)$ is calculated to assess how good an action is relative to the average. Common methods include some form of temporal difference estimation or generalized advantage estimation.

(3) Optimization of the Objective Function: A specialized objective function $J^{\theta^k}(\theta)$ is utilized, which can be approximated using a probability ratio. Equation (2) is the expression of objective function $J^{\theta^k}(\theta)$.

$$J^{\theta^k}(\theta) \approx \sum_{(s_t, a_t)} \frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)} A^{\theta^k}(s_t, a_t) \quad (2)$$

On the basis of this objective function, a KL penalty term is added, as shown in equation (3). The parameters are then updated using equation (3).

$$J_{PPO}^{\theta^k}(\theta) = J^{\theta^k}(\theta) - \beta KL(\theta, \theta^k) \quad (3)$$

(4) Optimization of the Objective Function: The strategy for updating the parameters is an adaptive KL penalty term, whose expression is given by equation (4).

$$\begin{cases} \beta \text{ increase, } KL(\theta, \theta^k) > KL_{max} \\ \beta \text{ decrease, } KL(\theta, \theta^k) < KL_{min} \end{cases} \quad (4)$$

The key feature of the PPO algorithm is its stabilization of the learning process by limiting the extent of policy updates. It uses a probability ratio to establish limits, avoiding drastic policy changes that might result from single data points, thus preventing significant drops in policy performance. Additionally, PPO allows for multiple policy updates with the same data in each iteration, increasing the efficiency of data usage.

C. Rewards Design

The Cauchy-distribution-based reward design method can autonomously update the influence coefficients of reward terms based on the reward values from the previous training session, eliminating the need for manual allocation of these coefficients. This approach ensures the maximization of the overall reward value and the successful completion of the training. It is assumed that there are n constraints imposed on the proposed actions for the task. Reward r_t includes: (1) reward terms that imposes constraints on actions from r_t^1 to r_t^n ; (2) a reward term that

judges action failure r_t^{n+1} ; (3) a reward term that determines action success r_t^{n+2} . The reward terms that impose constraints on actions include limitations on the force exerted on the workpiece, the force experienced by the robot, and the position and orientation constraints between the robot and the workpiece. These constraints are designed to ensure not only accurate movement positioning during robot operation but also to prevent damage to the robot itself or other workpieces and equipment (due to excessive contact force). Equation (5) is the expression of reward.

$$r_t = \sum_{i=1}^{n+2} \gamma_i r_t^i \quad (5)$$

Where γ_i is the influence coefficient of i -th reward term. It aims at balancing the effect of each reward term. Equation (6) shows that the sum of γ_i equals 1.

$$\sum_{i=1}^{n+2} \gamma_i = 1 \quad (6)$$

The initial value of γ_i is $\frac{1}{n+2}$. Finally, update the influence coefficient based on the overall reward r_t from the previous training session. Equation (7) shows the expression of updated influence coefficient.

$$\gamma_i' = \frac{r_t \times \gamma_i + \sigma_i}{\sum_{i=1}^{n+2} (r_t \times \gamma_i + \sigma_i)} \quad (7)$$

Where σ_i is update factor. It conforms to the Cauchy distribution: $\sigma_i \sim C(1, 0)$.

IV. RESULTS & EVALUATION

A. Experimental Simulation Scenario

Fig. 2 illustrates the simulated training environment for the peg-in-hole task constructed for the experiment. The robot selected is the Franka, which possesses seven degrees of freedom and is equipped with a gripper at its end-effector. Each degree of freedom and the gripper of the robot can be controlled independently. The combination of the simulation environment and workpiece models allows for on-demand modifications to the training scenario. Given the complexity of part collisions in the simulation environment, a Signed Distance Field (SDF) is employed for rendering instead of the more commonly used convex decomposition method. This approach has the advantage of accurately simulating the shapes of the hole and the peg, enabling precise physical collision simulation of the peg-in-hole process.

The reward design in the experimental peg-in-hole task is: the state vector s_t at each time t includes: the robot joint angles and gripper state vector θ_t , the peg-hole angle α_t , and the position and orientation vectors p_t of the peg and hole. The action vector consists of: the desired joint angles and gripper state vector θ_{t+1} for the next time step. In this task, we assume that the peg-hole angle, position, and orientation are obtainable information. Considering that the peg-hole angle and relative position significantly affect the final outcome during the peg-in-hole process, we define related reward terms r_t^1 and r_t^2 .

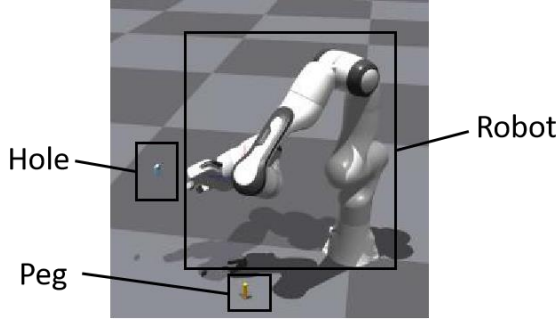


Fig. 2. Simulation scenario

Additionally, a penalty term r_t^3 is added to avoid incorrect insertion, and a reward r_t^4 is given for successful deep insertion of the peg. Equations (8) to (11) represent the reward expressions.

$$r_t^1 = 1 - e^{|\alpha_t|} \quad (8)$$

$$r_t^2 = -d_t \quad (9)$$

$$r_t^3 = \begin{cases} 0, & d_t \leq 0.5d_{hole} \\ -1, & 0.5d_{hole} \leq d_t \leq l_{hole} \\ -5, & d_t \geq l_{hole} \end{cases} \quad (10)$$

$$r_t^4 = \begin{cases} 1, & h_{peg} > h_{hole} - \delta \\ 0, & h_{peg} \leq h_{hole} - \delta \end{cases} \quad (11)$$

Where, d_t denotes the distance between the end of the peg and the centerline of the hole, d_{hole} represents the diameter of the hole, l_{hole} is the distance between holes, h_{hole} indicates the depth of the hole, h_{peg} signifies the depth of the peg inserted into the hole, and δ is the error threshold between the hole depth and the depth of the peg within the hole.

B. Effectiveness of Skill Learning Algorithm

The robot learns skills in a virtual training scenario: peg insertion. The PPO algorithm and the Deep Deterministic Policy Gradient (DDPG) algorithm were compared as reinforcement learning algorithms for skill acquisition. Each time the number of iterations increases by 10% of the total iterations, the success rate of the current policy in completing the task is evaluated. The success rate is calculated by attempting to complete the task 100 times with the same policy and determining the ratio of successful attempts to the total number of attempts. Fig. 3 illustrates the relationship between the success rate and the number of iterations for both skills. It can be observed that the PPO algorithm achieves a higher success rate for both tasks, demonstrating the stability and versatility of using the PPO algorithm as a skill learning algorithm.

C. Effectiveness of Skill Learning Platform

To validate the effectiveness of skill learning platform, we compared the traditional motion planning method with the skill learning approach utilizing virtual training environments. The robotic actions employed in the experiment were the peg

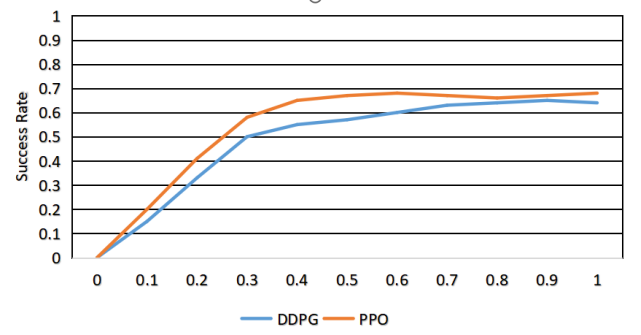
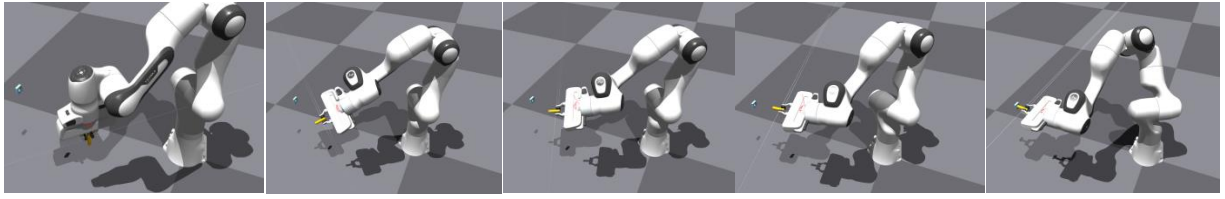


Fig. 3. Comparison of Skill Learning Success Rates Between DDPG and PPO

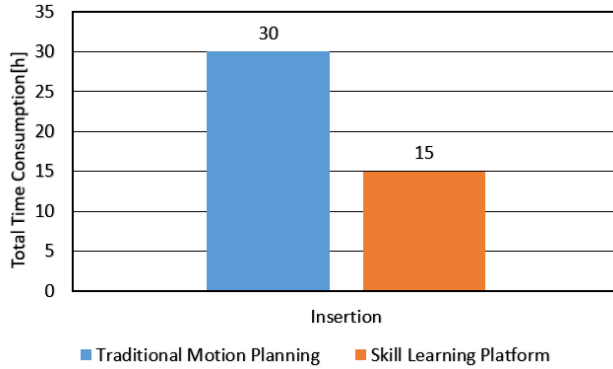
insertion, as depicted in Fig. 4(a). The comparison was made from the perspectives of development time and the reusability rate of development outcomes. Development time was compared not only in terms of the total time from development to training completion but also in terms of development time alone, to more accurately reflect personnel costs. The reusability rate refers to the proportion of previously developed outcomes, including assets and program code, that can be reused without modification or with minimal modification when handling a similar subsequent task. From the comparison results shown in Fig. 4(b) and (c), we found that training robots using virtual training environments can significantly reduce development time, while the deposition of development outcomes as "robotic skills" can greatly enhance the reusability of development outcomes.

V. CONCLUSION

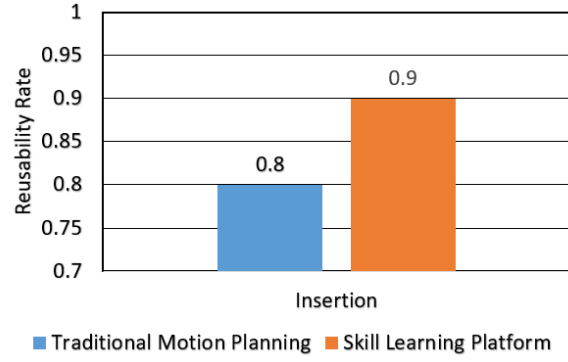
We proposed a robot skill learning system with a skill learning platform and a skill learning toolbox. The skill learning platform was able to construct simulation scenarios for robot skill learning, and the Docker-based skill learning toolbox enhanced the construction of simulation scenario. Proximal Policy Optimization (PPO) algorithm was served as the skill learning algorithm, and a Cauchy-distribution-based reward design method was proposed to facilitate effective robotic skill learning. A simulation peg-insertion scenario was constructed and selected as the testing scenario. The success rate of skill learning is increased due to our proposed Cauchy-distribution-based reward design method. The total program development time is reduced using our skill learning platform. The reusability rate of the robot program is increased by applying our proposed skill learning platform compared with traditional programming method. This robot skill learning system realizes robot skill learning in a simulation scenario using PPO algorithm with Cauchy-Distribution-based reward design. It reduces the development time, and increases the reusability rate of the skill.



(a) Insertion



(b) Comparison of Development Time



(c) Comparison of Reusability Rate

Fig. 4. Effectiveness Comparison of Skill Learning Platform and Traditional Motion Plan Method

REFERENCES

- [1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huang Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich, "Rt-1: Robotics transformer for real-world control at scale," arXiv:2212.06817, 2023.
- [2] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huang Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han, "Rt-2: Vision-language-action models transfer web knowledge to robotic 10 control," In Proceedings of The 7th Conference on Robot Learning, volume 229, pages 2165–2183, 06–09 Nov 2023.
- [3] Eric Jang, Alex Irpan, Mohi Khansari, Daniel Kappler, Frederik Ebert, Corey Lynch, Sergey Levine, and Chelsea Finn, "Bc-z: Zero-shot task generalization with robotic imitation learning," In Conference on Robot Learning, pages 991–1002. PMLR, 2022.
- [4] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, et al., "Scalable deep reinforcement learning for vision-based robotic manipulation," In Conference on Robot Learning, pp. 651–673. PMLR, 2018.
- [5] Tianhao Zhang, Zoe McCarthy, Owen Jow, Dennis Lee, Xi Chen, Ken Goldberg, and Pieter Abbeel, "Deep imitation learning for complex manipulation tasks from virtual reality teleoperation," In 2018 IEEE International Conference on Robotics and Automation (ICRA), pp. 5628–5635. IEEE, 2018.
- [6] Dmitry Kalashnikov, Jacob Varley, Yevgen Chebotar, Benjamin Swanson, Rico Jonschkowski, Chelsea Finn, Sergey Levine, and Karol Hausman, "Mt-opt: Continuous multi-task robotic reinforcement learning at scale," arXiv preprint arXiv:2104.08212, 2021a.
- [7] Eric Jang, Alex Irpan, Mohi Khansari, Daniel Kappler, Frederik Ebert, Corey Lynch, Sergey Levine, and Chelsea Finn, "Bc-z: Zero-shot task generalization with robotic imitation learning," In Conference on Robot Learning, pp. 991–1002. PMLR, 2021.
- [8] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, et al., "A generalist agent," arXiv preprint arXiv:2205.06175, 2022.
- [9] Mohit Shridhar, Lucas Manuelli, and Dieter Fox, "Cliport: What and where pathways for robotic manipulation," In Proceedings of the 5th Conference on Robot Learning (CoRL), 2021.
- [10] Mohit Shridhar, Lucas Manuelli, and Dieter Fox, "Perceiver-actor: A multi-task transformer for robotic manipulation," arXiv preprint arXiv:2209.05451, 2022.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin, "Attention is all you need," Advances in neural information processing systems, 30, 2017.
- [12] Yichi Zhang and Joyce Chai, "Hierarchical task learning from language instructions with unified transformers and self-monitoring," arXiv preprint arXiv:2106.03427, 2021.
- [13] Alexander Pashevich, Cordelia Schmid, and Chen Sun, "Episodic transformer for vision-and-language navigation," In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 15942–15952, 2021.
- [14] Simulately, "Overall Comparison", <https://simulately.wiki/docs/comparison>
- [15] J. Collins, S. Chand, A. Vanderkop and D. Howard, "A Review of Physics Simulators for Robotic Applications," in IEEE Access, vol. 9, pp. 51416–51431, doi: 10.1109/ACCESS.2021.3068769, 2021.

- [16] Mayank Mittal et al., "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740-3747, doi: 10.1109/LRA.2023.3270034, 2023.
- [17] Antonio Serrano-Muñoz, Dimitrios Chrysostomou, Simon Bøgh, Nestor Arana-Arexolaleiba, "skrl: Modular and Flexible Library for Reinforcement Learning," *Journal of Machine Learning Research*, vol. 24, no. 254, pp. 1-9, 2023.
- [18] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, Gavriel State, "Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning," *arXiv:2108.10470v2*, 2021.
- [19] O. M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba, "Learning dexterous in-hand manipulation," *Int. J. Robot. Res.*, vol. 39, no. 1, pp. 3-20, doi: 10.1177/0278364919887447, Jan. 2020.
- [20] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," *arXiv:1804.10332*. [Online]. Available: <http://arxiv.org/abs/1804.10332>, 2018.
- [21] A. K. Rath, D. R. Parhi, H. C. Das, M. K. Muni, and P. B. Kumar, "Analysis and use of fuzzy intelligent technique for navigation of humanoid robot in obstacle prone zone," *Defence Technol.*, vol. 14, no. 6, pp. 677 - 682, Dec. 2018. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S2214914718300229>
- [22] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pp. 31-36, Sep. 2017.
- [23] J. Chen, B. Jia, and K. Zhang, "Trifocal tensor-based adaptive visual trajectory tracking control of mobile robots," *IEEE Trans. Cybern.*, vol. 47, no. 11, pp. 3784-3798, Nov. 2017.
- [24] A. W. Winkler, C. D. Bellicoso, M. Hutter, and J. Buchli, "Gait and trajectory optimization for legged systems through phase-based endeffector parameterization," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1560-1567, Jul. 2018.
- [25] K. Takaya, T. Asai, V. Kroumov, and F. Smarandache, "Simulation environment for mobile robots testing using ROS and Gazebo," in *Proc. 20th Int. Conf. Syst. Theory, Control Comput. (ICSTCC)*, Oct. 2016, pp. 96-101.
- [26] G. Schoettler, A. Nair, J. A. Ojea, S. Levine and E. Solowjow, "Meta-Reinforcement Learning for Robotic Industrial Insertion Tasks," 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, pp. 9728-9735, doi: 10.1109/IROS45743.2020.9340848, 2020.
- [27] Fu, Zipeng and Zhao, Tony Z. and Finn, Chelsea, "Mobile ALOHA: Learning Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation," *arXiv:2401.02117v1*, 2024.
- [28] LanTianBaiYunDaHaiShaTanHerman, "PPO(Proximal Policy Optimization) Algorithm Theory and Implementation, A Detailed Explanation of Proximal Policy Optimization", https://blog.csdn.net/weixin_41106546/article/details/137359690
- [29] ReXueChuShiZhang, "Proximal Policy Optimization (PPO) Explained So Simply That Even a Dog Could Understand: A Detailed Guide to the PPO Algorithm", https://blog.csdn.net/weixin_42392454/article/details/140641453